Linpack and Other Benchmarks on Heterogeneous HPC Clusters

Nil Tianchen Mu
Arizona State University RTO Research Computing

RMACC HPC Symposium 2026 · Boise State University · May 12–14, 2026

Agenda

1. **ASU Sol cluster**
2. **HPL for Top500/Green500 with Nvidia Container**
3. **Other popular benchmarks overview**
4. **Pairwise OSU Micro-Benchmark design**
5. **NHC how-to**
6. **IO500 how-to**
7. **Q&A**

ASU Sol

Condominium cluster with two main GPU generations

Publicly funded baseline + privately purchased nodes on **one HDR InfiniBand fabric**, single ConnectX-7 NIC per node.

	A100 nodes	**H100 nodes**
Count	64 (public + private)	6 (private)
GPUs / node	4	2, 4, or 8
GPU mem	40 or 80 GB HBM2e	80 or 96 GB HBM3
FP64 peak / GPU	**19.5 TFlops**	**51 TFlops**
NVLink gen	3	4 or PCIe-only
CPU	AMD EPYC Zen 3, 48 c	Intel Xeon Sapphire Rapids, 48 / 96 / 104 c
System mem	512 GB	512 or 1024 GB

The Hypothesis We Started With

> *"If we add 3 H100 nodes (12 GPUs) to our 60 A100 nodes (240 GPUs), aggregate FP64 peak goes up **13 %**. The Top500 number should go up. Right?"*

However: our mixed run **dropped** Rmax by 38 % vs A100-only

Part 2

HPL for Top500/Green500 with Nividia HPC-Benchmarks Container

Top500 / Green500 Submission Overview

- **Top500** ranks the world's 500 fastest supercomputers by **HPL Rmax** (FP64 dense LU)
- **Green500** ranks HPL Rmax by average **GFlops / Watt**
- Published twice yearly: **ISC (June)**, **SC (November)**

What to submit

- `HPL.out` showing Rmax, N, NB, P×Q, residual
- System description (nodes, GPUs, IB, OS, MPI, BLAS)
- Theoretical Rpeak
- Green500: avg watts during a run, methodology level, instrumented fraction

```
================================================================================
T/V                N    NB     P     Q         Time          Gflops (   per GPU)
--------------------------------------------------------------------------------
WC0          1497600   512    15    16       870.21       2.573e+06 ( 1.072e+04)

HPL_pdgesv() start time Thu Oct 16 05:28:14 2025
HPL_pdgesv() end time   Thu Oct 16 05:42:44 2025

--------------------------------------------------------------------------------
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)=   0.000207094514 ...... PASSED
```

EEHPCWG Power Measurement Levels

	L1	L2	L3
Min instrumented	≥ **15 nodes** or **1/10** of system or **2 kW** (cap 40 kW)	≥ **15 nodes** or **1/8** of system or **10 kW**	**Entire system**
Meter accuracy	**±5 %**	**±2 %**	**±1 %**
Sampling	≥ 1 Hz, interval ≤ 10 % of core phase	≥ 1 Hz, **≥ 10 readings** in core phase	Energy meter, ≥ 10 readings
Reported	Core-phase avg	Core-phase + full-run avg + interval series	Total energy ÷ core-phase time
Idle power	Optional	Required	Required
Subsystems	Compute + interconnect	All participating	All participating

Submissions get downgraded if numbers don't match the level claimed.

Start from the NVIDIA HPC-Benchmarks Container

- **NGC catalog** (free): `nvcr.io/nvidia/hpc-benchmarks` — closed-source, NVIDIA-tuned binaries
- Binary **HPL, HPL-MxP, HPCG, STREAM** + `nccl-tests`, `nvshmem-perftest`, `osu-micro-benchmarks`, GEMM
- Matched **CUDA / NCCL / NVSHMEM / UCX / OpenMPI**
- We pinned `:25.09` for the Nov 2025 submission (current: `:26.02`)
- Example parameters sets for different GPU counts & type

Pull to a SIF once on shared storage, then exec from every node:

```
apptainer pull /scratch/sif/hpc-benchmarks_25.09.sif \
  docker://nvcr.io/nvidia/hpc-benchmarks:25.09

apptainer exec --nv -B /scratch:/scratch \
  /scratch/sif/hpc-benchmarks_25.09.sif \
  /workspace/hpl.sh --dat ./HPL.dat
```

Step 1: Validate One Node

Configuration	GPUs	N	NB	P×Q	**Rmax**	Rpeak	**Eff.**
1 A100 node	4	190,464	1024	2×2	**71.3 TFlops**	78.0 TFlops	**91.4 %**
1 H100 node (NVLink, 4-GPU)	4	190,464	1024	2×2	**179.7 TFlops**	204.0 TFlops	**88.1 %**

Per-GPU Rpeak: A100 = 19.5 TFlops · H100 = 51 TFlops (FP64 tensor-core basis)

- Run **single-node HPL** before anything else
- Rule of thumb: single-node efficiency **< 85 %** → fix it before scaling

Step 2: Two Nodes Tests

Preconditions — host driver match + Slurm cgroup

- Host `nvidia-smi` driver must be ≥ the driver baked into the container image
- Slurm `cgroup.conf` : set `ConstrainDevices=no` on GPU partitions.
- Default `yes` can block Apptainer's `--nv` from seeing all GPUs through the cgroup

Problem 1 — Heterogeneous IB device names

- `mlx5_0` , `mlx5_1` , `mlx5_ib0` not uniform across nodes
- Apptainer round-robins ranks across IB NICs

```
HCA=$(basename /sys/class/net/ib0/device/infiniband/*)
MAP=${HCA}:1
PE_MAP=$(printf "%s," $(yes "$MAP" | head -n "$GPU_N"))
```

Problem 2: Untangling UCX / NVSHMEM / NCCL

1. Build standalone **NVSHMEM** and **NCCL** test apps from NVIDIA's `multi-gpu-programming-models`

`Jacobi solver` exercises the same comm surface as HPL but iterates in seconds

2. Iterate env vars with `NCCL_DEBUG=INFO` / `NVSHMEM_DEBUG=INFO`

3. Port the working set to HPL, must use NVSHMEM

4. Reference:

- NCCL — https://github.com/NVIDIA/nccl-tests#
- NCCL — https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html
- NVSHMEM — https://docs.nvidia.com/nvshmem/api/gen/env.html
- HPL — https://docs.nvidia.com/nvidia-hpc-benchmarks/HPL_benchmark.html

Jacobi Solver Exercises

```
# dynamic HCA detection ...
# OpenMPI + UCX — reusable across any UCX-aware MPI app, not HPL-specific
export OMPI_MCA_pml=ucx  OMPI_MCA_coll=^ucc  OMPI_MCA_btl=^openib  PMIX_MCA_gds=hash
export UCX_TLS=dc_x,rc_x,sm,self,cuda_ipc,cuda_copy
export UCX_NET_DEVICES=$MAP  UCX_SOCKADDR_TLS_PRIORITY=rdmacm

# NCCL
export NCCL_NET=IB  NCCL_IB_HCA=$HCA  NCCL_SOCKET_IFNAME=ib0
export NCCL_COLLNET_ENABLE=1  NCCL_IB_PCI_RELAXED_ORDERING=1  NCCL_P2P_DISABLE=1
export NCCL_DEBUG=INFO

mpirun -n $GPU_N ./jacobi
```

NVSHMEM-only smoke test (run before re-adding NCCL):

```
NVSHMEM_HCA_PE_MAPPING=$PE_MAP NVSHMEM_DEBUG=INFO \
NVSHMEM_REMOTE_TRANSPORT=ibrc mpirun -n 4 ./jacobi
```

Runtime ENV Vars Tuning

```
# OpenMP
export OMP_NUM_THREADS=12
export OMP_PROC_BIND=close
export OMP_PLACES=cores

# UCX
export UCX_TLS=dc_x,sm,self,cuda_ipc,cuda_copy,gdr_copy
export UCX_NET_DEVICES=$MAP
export UCX_IB_GDR_DIRECT_RDMA=y

# NVSHMEM — must be on for the rest of the optimizations
export HPL_USE_NVSHMEM=1
export HPL_NVSHMEM_SWAP=1
export NVSHMEM_REMOTE_TRANSPORT=ibrc
export NVSHMEM_HCA_PE_MAPPING=$PE_MAP
export NVSHMEM_FORCE_IBGDA=1

# NCCL
export NCCL_NET=IB
export NCCL_COLLNET_ENABLE=1
export NCCL_IB_HCA=$HCA
export NCCL_IB_PCI_RELAXED_ORDERING=1
export NCCL_P2P_DISABLE=1   # disables PCIe-fallback P2P only — NVLink P2P still in use
```

Step 3: Tuning Knobs That Mattered

Three knobs in priority order:

1. GPU clock pinning

- 1410 MHz vs default 1395 MHz ≈ **+5 % Rmax**
- Thermally throttling nodes pinned to lower clocks (1275 / 1335 MHz)

```
wwctl ssh sg[001-999]  "nvidia-smi --lock-gpu-clocks=$min,$max"
```

2. Warm-up runs

- One small OOC run before the big one stabilizes boost clocks
- Run-to-run variance dropped from ±2 % to under ±0.5 %

3. Iterative parameter sweep on `N`, `NB`, `P × Q` — definitions and tuning advice on the next slide →

N × NB Sweep

Iterative bisection, not full grid — ~60 runs / 2 days

NB — block size of LU panel factorization

- **1024** sweet spot on tensor cores
- Textbook 32–256 is CPU HPL only

N — matrix dimension (problem size)

- Fill ~92 % of aggregate GPU memory: `N ≈ sqrt(0.92 × Σ GPU_mem / 8)`
- Round to a multiple of NB
- OOC mode (HPL_OOC_MODE=1) — spills matrix to host memory, allows N beyond GPU memory

P × Q — 2D MPI process grid

- **15 × 16** for 240 GPUs (nearest-square factor)
- P <= Q (row panel is serial bottleneck) → ~7 % over 16 × 15

The Mixed-Architecture Experiment

Configuration	GPUs	Rmax	Rpeak	Efficiency
2 A100 nodes	8	131.8 TF	156 TF	84.5 %
1 A100 + 1 H100 node	8	131.7 TF	282 TF	46.7 %
60 A100 nodes	240	**2,620 TF**	3,750 TF	**69.9 %**
60 A100 + 3 H100 nodes	252	**1,602 TF**	5,292 TF	**30.3 %**

Result: Adding 12 GPUs **dropped Rmax by ~1 PFlop**. Multiple problem sizes, NBs, P×Q grids tested — none recovered the loss.

Mixing GPU generations in vanilla HPL is a net loss, not a net gain.

Why the Mixed Run Failed

Load imbalance is part of the story — not all of it

- HPL distributes work uniformly; each iteration ends in a barrier
- H100 at **2.6× FP64** → H100 ranks finish early and wait at the barrier
- But imbalance alone caps efficiency near **~93 %** — we measured **30 %**

Other co-factors we suspect (still profiling):

- H100 **NIC asymmetry** — 4 of 8 GPUs cross SMP
- Cross-generation **NVLink collective tree** (gen 3 ↔ gen 4)
- **Host-memory BW saturation** in OOC at 252 GPUs

Heterogeneous-aware Options

- **SLATE** — distributed dense linear algebra, supports non-uniform tile sizes. Top500-eligible.
- **Patched HPL** — fork allowing non-uniform column counts in 2D block-cyclic distribution.
- Working with **NVIDIA HER + Benchmarks** teams on heterogeneous-aware container support.

Reproducibility: What to Keep, What to Publish

Scripts available ≠ results reproducible. Publish:

- **All HPL.dat files** used for the submission and the sweep
- **Raw Rmax logs** for every run — not just the best
- **Per-node** `nvidia-smi -ac` **clock settings** captured at job launch
- **NHC config**, **OSU helpers**, **NCCL test scripts** with thresholds
- **IPMI power CSVs** + the averaging script
- Pinned container digest, OFED version, driver version

Part 3

The Benchmark Landscape (Overview)

Popular HPC Benchmarks

Benchmark	Measures	Bottleneck	When to run
HPL	FP64 dense LU GFlops	Compute (matmul)	Top500 submission
HPL-MxP	Mixed-precision LU	Tensor cores	HPL-MxP list
HPCG	Sparse CG iterations	Memory BW + comm	Real-app proxy
STREAM	HBM / DRAM bandwidth	Memory channels	Per-node sanity
NCCL tests	Collective BW (allreduce…)	Interconnect + NCCL config	Pre-HPL, post-maintenance
OSU Micro-Benchmarks	Point-to-point BW & latency	NIC, IB, MPI stack	Per-pair health check
IO500	Storage BW + metadata	Filesystem, MDS	After storage changes
MLPerf HPC	Time-to-train AI workloads	Compute + comm + IO	AI-first sites

Part 4

Pairwise OSU Micro-Benchmarks Test

Test Design

0. Build OMB for each MPI implementation: OpenMPI, MVAPICH, Intel MPI
1. Generate node list input file from `sinfo`
2. Generate random disjoint node pairs, broad coverage at low cost
3. Submit one Slurm sbatch job per pair (not a job array)

`osu_bw` + `osu_latency`, CUDA-aware, device-to-device buffers

4. Repeat across iterations: Pair ≈ `1 - ((N-2)/N)^Round`
5. Check results & re-run

Flag any node whose median is **> 1.5× cluster median latency** OR **< 80 % cluster median BW**

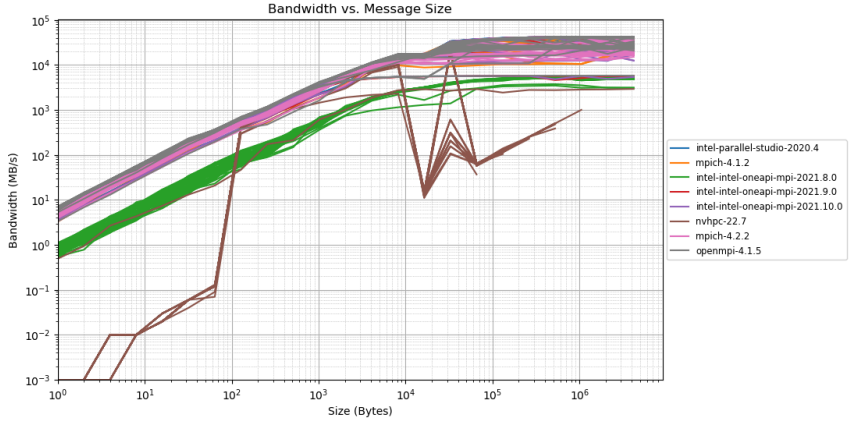
6. Plot all the pairwise results on one figure

CUDA-aware OMB Tests

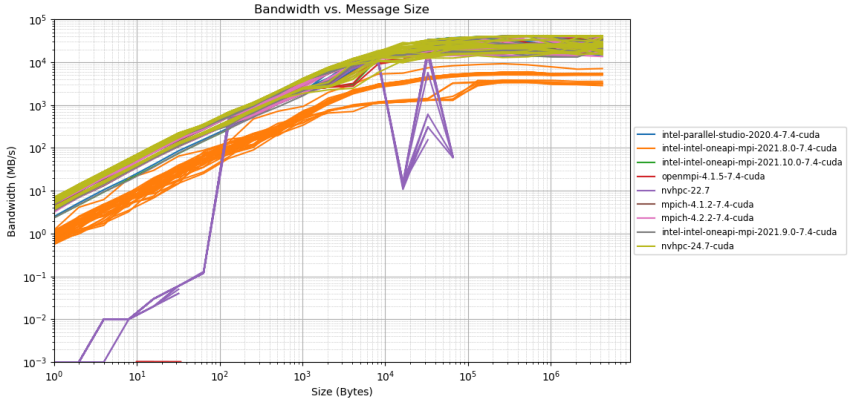
```
# 2 nodes, 1 GPU per rank, CUDA-aware OMB, device-to-device
srun -N 2 --ntasks-per-node=1 --gres=gpu:1 \
    osu_bw -d cuda D D

srun -N 2 --ntasks-per-node=1 --gres=gpu:1 \
    osu_latency -d cuda D D
```

- `-d cuda` — use CUDA-aware MPI buffers
- `D D` — both ranks allocate buffers on GPU device. Exercises GPUDirect RDMA
- Optional follow-up: `H H` (host-host), `D H` / `H D` (mixed) to localize where slowness lives

Pairwise OMB Bandwidth

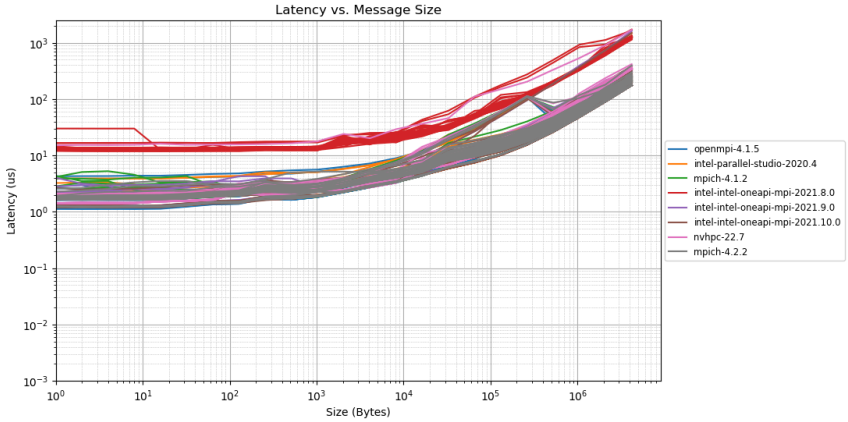


CPU bandwidth

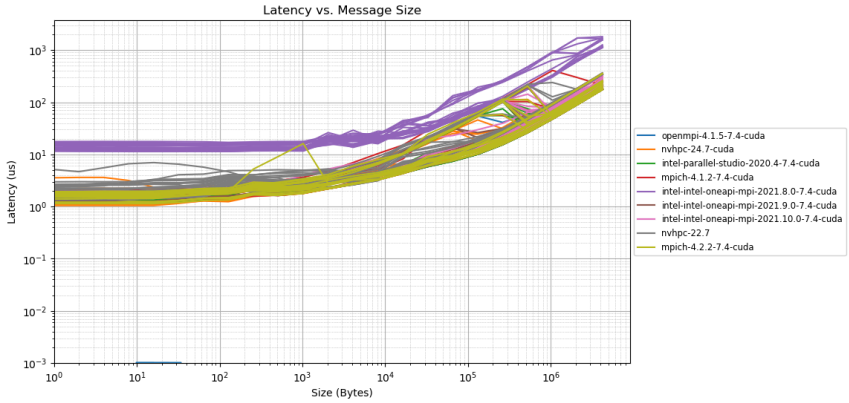


GPU bandwidth

Pairwise OMB Latency



CPU latency



GPU latency

Part 5

NHC how-to - Josh Burks

LBNL NHC

- `github.com/mej/nhc` — pure bash, no daemons
- slurmctld tells each node's slurmd to run:

```
HealthCheckProgram=/usr/sbin/nhc
HealthCheckInterval=300
HealthCheckNodeState=ANY,CYCLE
```

- On failure: `scontrol update nodename=… state=DRAIN reason="NHC: <check>"`
- `/etc/nhc/nhc.conf`

```
*  || export PATH=/usr/sbin:/usr/bin:/sbin:/bin
*  || check_hw_cpuinfo 2 96 96
*  || check_hw_physmem 510GB 530GB 1%
*  || check_fs_mount_rw -f /scratch -t lustre
*  || check_fs_mount_rw -f /home    -t nfs
*  || check_cmd_output -t 10 -c 0 -m '/4 GPUs/' "nvidia-smi -L | wc -l"
*  || check_dmesg -E 'Xid|MCE|EDAC'
*  || check_ps_loadavg 96
```

Part 6

IO500 how-to - Alan Chapman

IO500 Overview

- Published twice yearly (ISC + SC)
- **IOR** (bandwidth) + **mdtest** (metadata) + a `find` test
- Each phase has an **easy** variant (you tune) and a **hard** variant (fixed params)
- **10-Node Challenge** — separate list for small clusters, fits campus sites
- **Stonewall** caps each phase at the same wall-time → fair scoring
- **Score** = geometric mean of bandwidth and metadata phase results

Our IO500 Test Run

- POSIX-only to reflect real usage
- **10 nodes × 4 tasks = 40 MPI ranks** (10-Node Challenge), OpenMPI 4.1.5
- `srun ./io500 config.ini`
- Filesystem: **3,722 TiB · 78 % full**
- **No Lustre stripe tuning** — defaults reflect what users actually see

Summary

- Top500 / Green500 HPL Runs with Nvidia HPC-Benchmarks Container
- Popular benchmarks
- Pairwise OSU Micro-Benchmarks Tests
- NHC
- IO500

Thank You!

ASU RTO Research Computing:

Nil Tianchen Mu · Josh Burks · Alan Chapman · Juanjo García Mesa

ASU RTO Computational Research Accelerator, Director: Gil Speyer

NVIDIA HER (A. Butler, S. Niemi) and Benchmarks (M. Almasri) teams

`nil.mu@asu.edu`

github.com/NilaBlueshirt/HPC_benchmarking

made with slidev